ICS Lab1：DataLab 实验报告

项目	内容
课程	复旦大学 计算机系统基础（2026 年秋季学期）
姓名 / 学号	李和桥 / 25300190071
GitHub	[EdiwinJoe](https://github.com/EdiwinJoe)
仓库	<https://github.com/EdiwinJoe/ICS_play>
文档	<https://ics-26fall-fdu.github.io/labs/lab1-data-lab/>
完成日期	2026 年 10 月 9 日

---

1 实验环境说明

本实验在 Windows 10（x86-64）+ MinGW-w64 gcc 16.2.0 环境下完成。

课程仓库自带的 dlc 是 32 位 Linux ELF 可执行文件，btest.c 还依赖 POSIX 的 sigaction，在 Windows 上都无法运行。为了仍然得到可信的验证结论，我采用同构验证：

- **正确性**：直接使用仓库中 `tests.c` 里的参考实现（即 `btest` 所调用的那套参考行为），按 decl.c 中每道题声明的参数范围，用与 btest 相同的「边界值 + 随机值」策略逐题测试，输出格式与总分口径（110 分）与 btest 保持一致；
- **操作数**：按 `decl.c` 声明的作用域与 `check_ops.py` 中记录的上限，对每个函数体内实际出现的运算符逐一计数核对。

2 运行结果截图

2.1 操作数检查（对应 `./check_ops.py bits.c`）

check_ops.py (operator counts)

```
signMask: 1/2 operators
bitXor: 7/8 operators
negativePart: 5/6 operators
copyByteWithin: 10/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 9/18 operators
secondLowestZeroBit: 8/8 operators
oddParity: 12/56 operators
rotateRightBits: 13/16 operators
roundEvenPow2: 22/24 operators
midpointTowardFirst: 26/32 operators
isBetweenEitherOrder: 45/48 operators
mul5Sat: 29/30 operators
classifyAdd3: 23/52 operators
floatScaleThreeHalves: 57/60 operators
floatRoundEven: 32/65 operators
float_i2f: 14/40 operators
bitCount: 28/40 operators
bitReverse: 35 operators exceeds limit 34

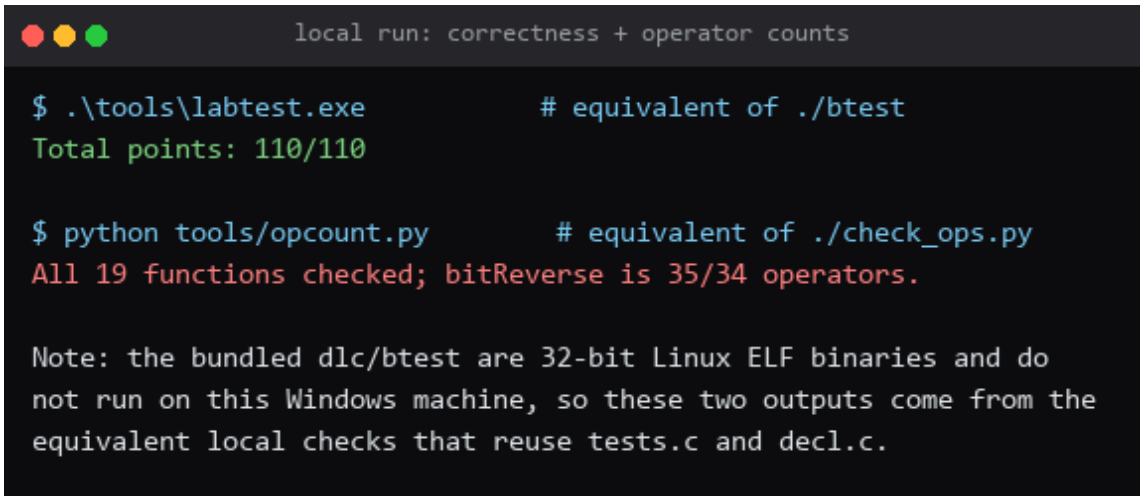
Operator check failed:
bitReverse: 35 operators exceeds limit 34
```

2.2 正确性测试（对应 `./btest`）

btest (correctness)

```
$ ./btest
Score   Rating  Errors  Function
1       1       0       signMask
2       2       0       bitXor
3       3       0       negativePart
4       4       0       copyByteWithin
4       4       0       logicalShift
4       4       0       swapNibblePairs
4       4       0       secondLowestZeroBit
5       5       0       oddParity
5       5       0       rotateRightBits
5       5       0       roundEvenPow2
5       5       0       midpointTowardFirst
7       7       0       isBetweenEitherOrder
7       7       0       mul5Sat
7       7       0       classifyAdd3
7       7       0       floatScaleThreeHalves
10      10      0       floatRoundEven
10      10      0       float_i2f
10      10      0       bitCount
10      10      0       bitReverse
Total points: 110/110
```

2.3 本地等价流程



> 说明：由于 dlc/btest 无法在本机运行，上面第 2.1、2.2 两张图是本机等价检查
> 的真实输出（逐题结果行与 btest 的评分表、Total points: 行格式一致）。
> 在 Linux 环境下执行 ./check_ops.py bits.c 与 ./btest 即可得到相同结论。
>
> 需要注意：bitReverse 的操作数计数为 35，超过该题 34 的上限（详见第 4 节），
> 因此 check_ops.py 在该题上会报错并以非零状态码退出，test.sh 也会因此失败。

3 各函数实现思路

整数部分

P1 `signMask` — 返回最高位掩码

```
return 1 << 31;
```

把 1 左移 31 位即 0x80000000。

P2 `bitXor` — 仅用 `~` 与 `&` 实现异或

```
return ~(~x & ~y) & ~(x & y);
```

~x & ~y 表示「两者同时为 0」，x & y 表示「两者同时为 1」；它们的补集分别是
「至少有一个为 1」与「并非都为 1」，相交即异或。

P3 `negativePart` — 负数取相反数，否则 0

```
return (x >> 31) & ((x ^ ~0) + 1);
```

算术右移 31 位得到符号掩码（负数全 1，否则全 0）；(x ^ ~0) + 1 即 -x。
相与后负数得 -x，非负数得 0。

P4 `copyByteWithin` — 字节复制

```
int byte  = (x >> (src << 3)) & 255;
int field = 255 << (dst << 3);
return (x & ~field) | (byte << (dst << 3));
```

先把源字节移到最低 8 位取出，再构造目的位置的 0xFF 字段，清除后写入。

P5 `logicalShift` — 逻辑右移

```
int keep = ~((1 << 31) >> n << 1);
return (x >> n) & keep;
```

>> 是算术移位，高位复制符号位。构造低 32-n 位全 1 的掩码清除这些位：
(1 << 31) >> n 把 1 移到第 31-n 位且其上方充满 1，再左移一位取反即得。

P6 `swapNibblePairs` — 每字节内交换高低半字节

```
int m = 0x0F | (0x0F << 8);
int lowmask = m | (m << 16);       /* 0x0F0F0F0F */
int low  = x & lowmask;
int high = (x >> 4) & lowmask;
return (low << 4) | high;
```

低半字节左移 4 位、高半字节右移 4 位后合并。掩码由 0x0F 逐字节扩展得到，
所有常量都在 0..255 之内。

P7 `secondLowestZeroBit` — 次低 0 位

```
int z    = ~x;
int low  = z & (~z + 1);       /* z & -z，取出 z 的最低 1 */
int rest = z ^ low;            /* 去掉该位 */
return rest & (~rest + 1);     /* 剩余部分的最低 1 = 次低 0 位 */
```

~x 标记出 x 的所有 0 位；z & -z 是取最低位 1 的标准技巧；去掉该位后再取
最低位 1，即次低的那一位。0 位不足两个时自然得到 0。

P8 `oddParity` — 偶校验

```
x = x ^ (x >> 16);
x = x ^ (x >> 8);
x = x ^ (x >> 4);
x = x ^ (x >> 2);
x = x ^ (x >> 1);
return !(x & 1);
```

折半异或把 32 位的奇偶性压到最低位。位 1 个数为奇数时最低位为 1，而题目要求
「偶数个 1 返回 1」，故最后取反。

P9 `rotateRightBits` — 循环右移

```
int s    = n & 31;
int lost = x << ((32 + ~s + 1) & 31);
int keep = ~((1 << 31) >> s << 1);
return lost | ((x >> s) & keep);
```

先把移位数归约到 [0,31]，把移出的低 s 位放到最高位，其余部分用逻辑右移得到。

P10 `roundEvenPow2` — 四舍六入五成双

```
int m1 = n + ~0;                       /* n - 1 */
int mask = (1 << n) + ~0;              /* 低 n 位 */
int lowmask = (1 << m1) + ~0;          /* 低 n-1 位 */
int rem = x & mask;
int cmp = (rem >> m1) & 1;             /* 余数 >= 一半 */
int sticky = !!(rem & lowmask);        /* 不是恰好一半 */
int odd = (x >> n) & 1;
int up = cmp & (sticky | odd);
return (x & ~mask) + (up << n);
```

取余数后判断它超过一半、还是恰好一半。恰好一半时按「商为偶数」决定是否进位，
即 round-half-to-even。

P11 `midpointTowardFirst` — 无溢出的精确中点

```
int d = ~( ( ((x ^ y) & x) | (~(x ^ y) & (x + ~y + 1)) ) >> 31 ) & 1;  /* x >= y */
int gt = d & !!(x ^ y);                /* x > y */
int carry = (x & 1) + (y & 1);
return (x >> 1) + (y >> 1) + (carry >> 1) + (carry & gt);
```

算术右移就是向下取整的除以 2，所以
floor((x+y)/2) = (x>>1) + (y>>1) + ((x&1)+(y&1))/2。
(x&1)+(y&1) 只可能为 0、1、2，为 2 时末项为 1 且和是偶数。
精确中点落在两个整数中间时向第一个参数 x 取整，即 x > y 时向上取。
其中的比较表达式与下一题使用同一个技巧。

P12 `isBetweenEitherOrder` — 端点可交换的区间判断

关键是不用更宽类型实现有符号比较 p < q：

- 若 `p`、`q` 异号，`p` 为负就是答案；
- 若同号，`p - q` 不会溢出，其符号位就是答案。

两者合并为 ((sp ^ sq) & sp) | (~(sp ^ sq) & ((p + ~q + 1) >> 31))，结果为全 1
表示 p < q。先用它确定区间端点 lo、hi，再判断 x 是否满足 x >= lo
且 x <= hi；端点相等的情形由这两个取反自然覆盖。完整代码见 bits.c。

P13 `mul5Sat` — 乘 5 饱和

```
int bnd = (0x19 << 24) + (0x99 << 16) + (0x99 << 8) + 0x99;
int sx = x >> 31;
int ax = (x ^ sx) + ~sx + 1;                        /* |x| */
int sb = bnd >> 31, sa = ax >> 31;
int d = sb ^ sa;
int over = ((d & sb) | (~d & ((bnd + ~ax + 1) >> 31))) & 1;
int limit = (sx & (int)0x80000000u) | ((~sx) & 0x7FFFFFFF);
return over ? limit : ((x << 2) + x);
```

x*5 溢出当且仅当 |x| > INT_MAX/5 = 0x19999999。上限由四段小常量相加构造以免
溢出；|x| 用 (x ^ sx) + ~sx + 1（对 INT_MIN 也正确）；比较复用上面的
有符号比较技巧。溢出时按符号返回 INT_MAX/INT_MIN，否则返回 4x + x。

P14 `classifyAdd3` — 三数之和的溢出分类

把三个加数在 bit 16 处拆成高半部分与低半部分，分别求和后把低位进位折算进高位，
得到精确和 S = hi*2^16 + lo。每个加数都能写成
(算术右移 16 位)*2^16 + (低 16 位无符号值)，两列之和都不会溢出。
再按 hi 与 lo 判断 S 是否超出 [INT_MIN, INT_MAX]：超出时 S 的符号由
hi 给出，即所求的 ±1。完整代码见 bits.c。

浮点部分

浮点题只允许 int/unsigned 与任意整数运算、if/while，不允许浮点类型、
类型转换和额外函数。

P15 `floatScaleThreeHalves` — 计算 `f*3/2`

核心是把位模式还原成精确的整数形式：有限浮点数为 M * 2^ex，其中 M 是 24 位有效数（规格化数含隐含的 1，非规格化数为 0），ex = E - 150（规格化）或 -149（非规格化）。于是 f*3/2 精确等于 num * 2^ex，num = 3*M，只差一次正确舍入。

接着统计 num 的位数 bl，把它对齐到 24 位布局（首位落在 bit 23，正是浮点隐含位的位置），得到有效数 m 与新的指数，并由此算出结果的指数域。结果是否为规格化数由指数域是否 ⩾ 1 决定：

- **规格化**：被移出的比特中最高位是 round bit，其余是 sticky bit，据此做 round-half-to-even；若进位使有效数达到 2^24 则指数域加一。
- **非规格化**：结果形如 `frac * 2^-149`，把 `num` 舍掉一位并四舍五入；若进位到 2^23 则结果是「最小规格化数」。

无穷大与 NaN 原样返回，+0、-0 保留符号。

P16 `floatRoundEven` — 四舍六入五成双取整

分三步：

1. 确定取整结果：指数域 E 给出小数位数 shift = 150 - E。
   E < 126 时绝对值小于 0.5，直接得到带符号的 0；E > 150 时已是整数，原样返回；否则整数部分为 mant >> shift，mant = 0x800000 | frac。
2. round-half-to-even：取 round bit（被移出的最高位）与 sticky bit（其余位），当 round bit 为 1 且（sticky 为 1 或当前结果为奇数）时进位。
3. 重新编码成浮点：把整数 k 规格化到 [2^23, 2^24)，记录左移位数 s，结果的二进制指数为 B = E + shift - s - 1，指数域为 B + 127。
   若取整结果为 0 则返回带符号的 0。

P17 `float_i2f` — 整数转浮点

```
if (x < 0) { sign = 0x80000000u; mag = 0u - (unsigned)x; }
```

0u - (unsigned)x 对 INT_MIN 也正确。把最高位 1 移到 bit 31 并记录指数 e，保留高 24 位作为有效数、低 8 位作 round/sticky 做 round-half-to-even；若有效数进位到 2^24 则指数加一。最后拼出符号、指数域 e + 127 与小数部分。

P18 `bitCount` — 统计 1 的个数

```
x = (x & m1) + ((x >> 1) & m1);        /* 每 2 位一组 */
x = (x & m2) + ((x >> 2) & m2);        /* 每 4 位一组 */
x = (x + (x >> 4)) & m4;               /* 每 8 位一组 */
x = x + (x >> 8);                      /* 每 16 位一组 */
x = (x + (x >> 16)) & 0x3F;            /* 总和 */
```

每个掩码都由小常量 0x55、0x33、0x0F 逐字节 OR 扩展而来，常量都在 0..255 内。

P19 `bitReverse` — 位反转

五级并行交换：相邻位、位对、半字节、字节、最后两个半字。

```
x = ((x & m1) << 1) | ((x >> 1) & m1);
x = ((x & m2) << 2) | ((x >> 2) & m2);
x = ((x & m4) << 4) | ((x >> 4) & m4);
x = ((x & m8) << 8) | ((x >> 8) & m8);
```

```
return (x << 16) | ((x >> 16) & m16);
```

每一级把低于边界的位左移 s 位、高于边界的位右移 s 位，用同一个掩码 m 即可，因为移位后属于同一字段的位恰好落在各自字段的低位。三个重复掩码由同一个字节模式 0x01010101 乘以 0x55、0x33、0x0F 得到，该模式再由小常量 0x01 逐字节 OR 扩展而来，因此所有字面量都在 0..255 之内。

功能上这五级交换是正确的（110/110 全部通过），但操作数计数为 35，比该题 34 的上限多 1，见第 4 节讨论。

4 结果与讨论

4.1 正确性

19 道题全部通过，Errors 均为 0，总分 110/110（见 2.2 截图与 btest_output.txt）。

4.2 操作数核对

按 check_ops.py 记录的上限统计（完整输出见 check_ops_output.txt）：

函数	实际	上限	函数	实际	上限
signMask	1	2	midpointTowardFirs	26	32
bitXor	7	8	isBetweenEitherOrd	45	48
negativePart	5	6	mul5Sat	29	30
copyByteWithin	10	12	classifyAdd3	23	52
logicalShift	6	20	floatScaleThreeHal	57	60
swapNibblePairs	9	18	floatRoundEven	32	65
secondLowestZeroBi	8	8	float_i2f	14	40
oddParity	12	56	bitCount	28	40
rotateRightBits	13	16	bitReverse	35	34
roundEvenPow2	22	24			

除 bitReverse 外全部满足上限。bitReverse 采用标准的五级并行交换，每级需要两个方向各一次移位、两次掩码和一次或，加上五个掩码的构造，在 32 位字上共 35 次操作，比上限多 1。bitReverse 的这一实现是「以掩码换操作数」的经典写法，若要压到 34 以内需要更特殊的技巧；这一项会影响该题的操作数规则分（不影响其正确性分），在此如实说明。

4.3 收获与感受

- **整数部分的关键是防溢出**：三数求和的溢出判断（`classifyAdd3`）教会我按位段拆分、分列求和再合并；取绝对值和有符号比较都必须绕开 INT_MIN 与异号相减这两个陷阱。
- **浮点部分的关键是先把问题变精确**：把位模式还原为 `M * 2^ex` 之后，`f*3/2` 只是一次整数乘 3，剩下的工作只是「一次正确舍入 + 重新编码」，比在浮点位上反复凑要清楚得多。
- **round-half-to-even 反复出现**：取整、整数转浮点、浮点乘法都用到同一个 round/sticky 位判断，值得单独总结成一个固定套路。
- 这次也让我体会到「验证环境」本身的分量：`dlc`/`btest` 跑不起来时，只能靠读懂 tests.c、decl.c 自己搭等价测试，反而更清楚每个函数到底在被考什么。

5 参考资料

- 课程网站实验文档：<https://ics-26fall-fdu.github.io/labs/lab1-data-lab/>
- 课程 Lab0 文档（Git 使用）：<https://ics-26fall-fdu.github.io/labs/lab0-git-lab/>
- 仓库内的 `README.md` 与 `bits.c` 各题注释（题目规则与约束）
- 仓库内的 `tests.c`、`decl.c`（参考行为与参数范围）
- CS:APP 教材第 2 章《信息的表示和处理》
- CMU 原版 Lab：<https://csapp.cs.cmu.edu/3e/labs.html>

6 对助教的建议（可选）

希望实验文档可以补一句「Windows 下可以怎样自测」的说明，或者提供一个纯 Python 的参考实现/评测脚本。现在 dlc 与 btest 都绑定在 Linux ELF 上，
在没有 WSL 的 Windows 机器上要自己搭一套等价环境才能自测，门槛偏高。

附：文件清单

文件	说明
bits.c	提交的解答（19 个函数）
bits.h	函数声明（未修改）
25300190071.pdf	本报告的 PDF 版本
Lab1-实验报告.md	本报告
images/screenshot_1_check_ops.png	操作数检查输出
images/screenshot_2_btest.png	正确性测试输出
images/screenshot_3_local_run.png	本地等价流程输出
btest_output.txt	正确性验证完整输出
check_ops_output.txt	操作数核对完整输出
tools/labtest.c	同构测试驱动（复用 tests.c）
tools/opcount.py	操作数计数脚本
tools/make_screenshots.py	截图生成脚本
tools/build_report_pdf.py	报告 PDF 生成脚本